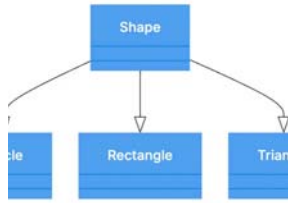


B 3.2.1 Inheritance in OOP Python (HL Only)



B3.2.1 Explain and apply the concept of inheritance in OOP to promote code reusability. (HL Only)

This section explores inheritance in OOP for code reusability. It explains hierarchical relationships between parent and child classes, how to extend classes, and the impact of access modifiers (private, public, protected, default) on inherited members.

Inheritance in Python: Building Hierarchies and Reusing Code

Inheritance is a fundamental pillar of OOP that allows a new class (the **child class** or **subclass**) to inherit properties and behaviours from an existing class (the **parent class** or **superclass**). This mechanism establishes a hierarchical relationship between classes, where the child class is a more specific version of the parent class.

Hierarchical Relationships Between Parent and Child Classes

Think of real-world categories and subcategories. For instance, "Animal" is a broad category, and "Dog" and "Cat" are more specific types of animals. In OOP, we can model this relationship using inheritance. The **Animal** class can be the parent class, and **Dog** and **Cat** can be child classes that inherit general characteristics of animals, such as having a name and being able to eat. They can also have their own specific characteristics, like a dog's ability to bark or a cat's ability to purr.

In Python, we define a child class by including the parent class name in parentheses after the child class name in the class definition. Let's look at a simple example:

```

# Parent class: Animal
class Animal:
    def __init__(self, name):
        self.name = name
        print(f"Animal created: {name}")

    def eat(self):
        print(f"{self.name} is eating.")

# Child class: Dog
class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name) # Call the constructor of the parent class
        self.breed = breed
        print(f"Dog created: {name}, breed: {breed}")

    def bark(self):
        print(f"{self.name} is barking.")

# Example usage
my_dog = Dog("Buddy", "Golden Retriever")
my_dog.eat() # Inherited method from the Animal class
my_dog.bark() # Method specific to the Dog class

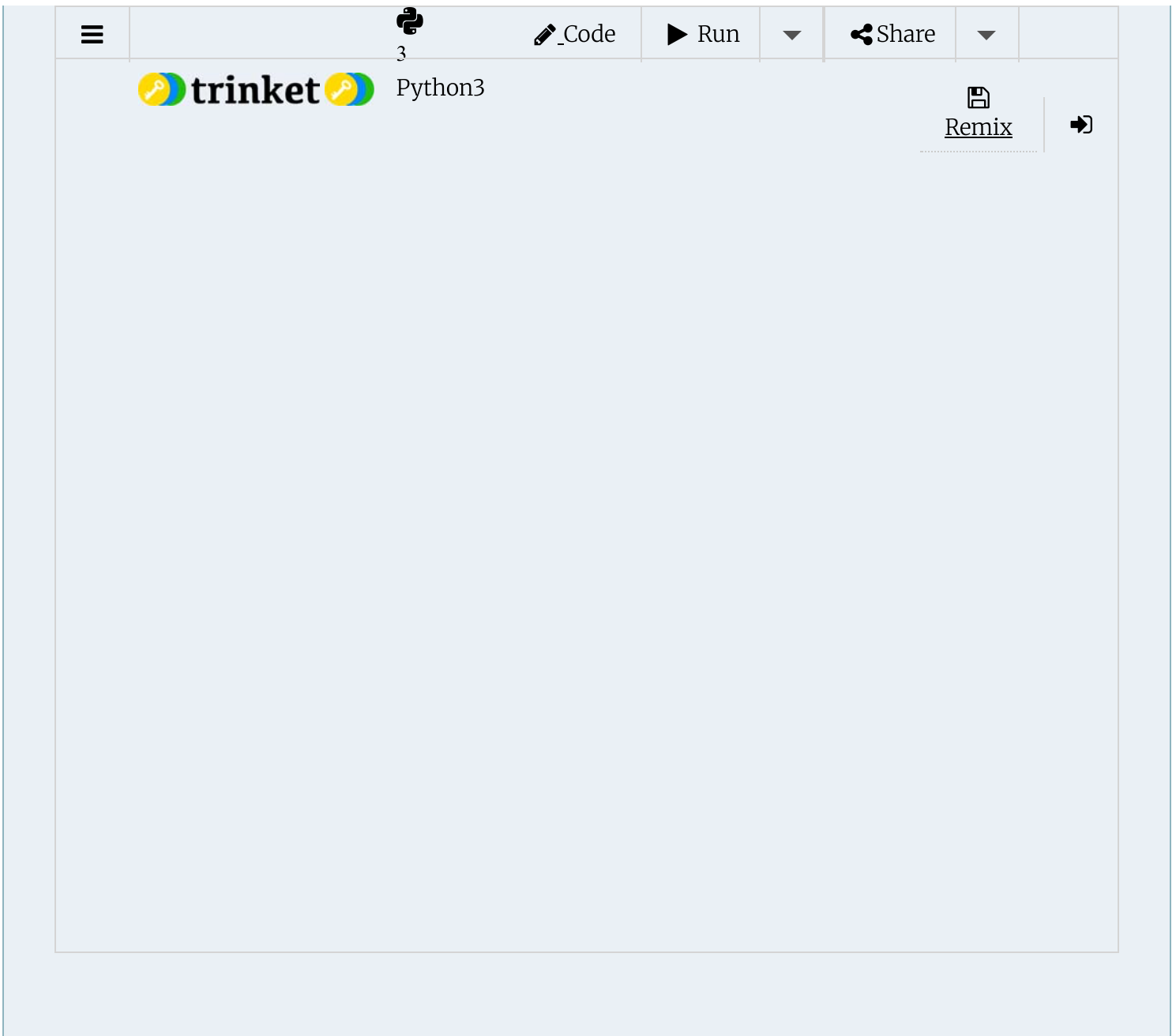
```

In this code:

- The `Dog` class is defined as `class Dog(Animal):`, indicating that it **inherits** from the `Animal` class.
- The `Dog` class inherits the `__init__` method (constructor) and the `eat()` method from `Animal`.
- The `Dog` class also has its own specific attribute (`breed`) and method (`bark()`).
- The `super().__init__(name)` call in the `Dog` constructor is crucial. It invokes the `__init__` method of the `Animal` parent class to initialize the inherited `name` attribute. This ensures that the parent class's initialisation logic is executed.

Try it out

Try instantiating another `Dog` and calling on its behaviours:



Extending Existing Classes to Reuse and Extend Functionalities

The primary benefit of inheritance is **code reusability**. Instead of rewriting common attributes and methods in multiple classes, you can define them once in a parent class and then have child classes inherit them. This leads to:

- **Reduced code duplication:** Less code to write and maintain.
- **Increased maintainability:** Changes made to the parent class are automatically reflected in its child classes (where applicable and not overridden).
- **Improved organisation:** A Clearer hierarchical structure of classes makes the code easier to understand and manage.

Furthermore, child classes can **extend** the functionality of the parent class by:

- **Adding new attributes and methods:** As seen with the `breed` and `bark()` in the `Dog` class.
- **Overriding inherited methods:** A child class can provide its own specific implementation of a method that it inherited from its parent class. For example, if we had a `make_sound()` method in `Animal`, both `Dog` and `Cat` could override it to produce their respective sounds.

"Access Modifiers" and Inheritance in Python

Python has a slightly different access control approach than languages like Java. It relies more on convention than strict enforcement, using keywords like `public`, `private`, and `protected`. However, there are naming conventions that indicate the intended scope of a member:

- **Public:** Members (attributes and methods) with no leading underscores are considered public and can be accessed from anywhere, including child classes.

```
class Parent:
    def __init__(self):
        self.public_var = 10

    def public_method(self):
        print("Public method in Parent")

class Child(Parent):
    def access_parent_members(self):
        print(f"Accessing public_var from Child: {self.public_var}")
        self.public_method()
```

- **"Protected":** Members with a single leading underscore (e.g., `_protected_var`) are conventionally treated as protected. Ideally, they should only be accessed within the class and its subclasses. However, Python does not prevent external access to these members; it's more of a programmer's agreement.

```
class Parent:
    def __init__(self):
        self._protected_var = 20

    def _protected_method(self):
        print("Protected method in Parent")

class Child(Parent):
    def access_protected(self):
        print(f"Accessing _protected_var from Child: {self._protected_var}")
        self._protected_method()

# Outside the class, you can still access _protected_var, though it's discour:
parent = Parent()
print(parent._protected_var)
```

- **"Private":** Members with double leading underscores (e.g., `__private_var`) have a mechanism called "name mangling." Python renames these attributes to make them harder to access from outside the class. While not truly private in the strict sense, it discourages accidental external access and provides a stronger indication of intended privacy. However, they are still accessible within the class and can be accessed from subclasses using the mangled name (`_Parent__private_var` in this case).

```

class Parent:
    def __init__(self):
        self.__private_var = 30

    def __private_method(self):
        print("Private method in Parent")

    def access_private_from_parent(self):
        print(f"Accessing __private_var from Parent: {self.__private_var}")
        self.__private_method()

class Child(Parent):
    def try_access_private(self):
        # print(f"Trying to access __private_var from Child: {self.__private_}
        # self.__private_method() # This would also raise an AttributeError
        super().access_private_from_parent() # Access through the parent's me
        print(f"Accessing mangled __private_var from Child: {self._Parent__pr
        self._Parent__private_method()

# Outside the class
parent = Parent()
# print(parent.__private_var) # This would raise an AttributeError
print(parent.access_private_from_parent())

```

In summary, Python uses naming conventions to suggest the intended scope of class members in the context of inheritance. Single underscores suggest protected access (accessible by subclasses), and double underscores trigger name mangling to make access from outside the class more difficult, though not impossible. Public members have no leading underscores and are freely accessible.

Summary

Inheritance in Python is a powerful tool for creating hierarchical class structures and facilitating code reuse. You can avoid redundant code and build upon existing functionalities by defining child classes that inherit from parent classes. Python utilises a convention-based approach to manage the accessibility of parent class members in child classes. Members without leading underscores are public, a single leading underscore suggests protected access (intended for use by subclasses), and double leading underscores trigger name mangling to make external access less

straightforward. Understanding these conventions is key to designing well-structured and maintainable object-oriented programs in Python.

All materials on this website are for the exclusive use of teachers and students at subscribing schools for the period of their subscription. Any unauthorised copying or posting of materials on other websites is an infringement of our copyright and could result in your account being blocked and legal action being taken against you.

 **Report a problem**